devpixelforge

Procesamiento de imágenes: Rust (binario nativo) + Go (bridge FFI). Binario optimizado con LTO y codegen-units=1.

How to Use

bash

```bash
# Build completo
make build

# Verificar instalación
./dpf/target/release/dpf caps

# Procesar imagen
./dpf/target/release/dpf process \
  --job '{"operation":"resize","input":"img.png","output_dir":"/tmp","widths":[320,640]}'
```

Skills

Skill	Trigger	Path
`rust-best-practices`	Rust code review, idioms, ownership	`.agents/skills/rust-best-practices/`
`golang-pro`	Concurrencia, gRPC, microservicios	`.agents/skills/golang-pro/`

🎯 Model Assignment (Sub-Agent Configuration)

Configuración de modelos establecida para este proyecto:

Phase	Modelo	Rol
`sdd-orchestrator`	OpenCode Go / Kimi K2.5	Coordinator
`sdd-init`	OpenCode Go / MiniMax M2.5	Init
`sdd-explore`	OpenCode Go / MiniMax M2.5	Exploración
`sdd-propose`	OpenCode Go / Kimi K2.5	Propuestas arquitectónicas
`sdd-spec`	OpenCode Go / MiniMax M2.5	Especificaciones
`sdd-design`	OpenCode Go / Kimi K2.5	Diseño técnico
`sdd-tasks`	OpenCode Go / MiniMax M2.5	Breakdown
`sdd-apply`	OpenCode Go / MiniMax M2.5	Implementación
`sdd-verify`	OpenCode Go / MiniMax M2.5	Verificación

Phase	Modelo	Rol
`sdd-archive`	GitHub Copilot / Claude Haiku 4.5	Archivado
`default`	OpenCode Go / MiniMax M2.5	Tareas generales

Notas:

- Kimi K2.5 para fases que requieren razonamiento (propose, design)
- MiniMax M2.5 para tareas mecánicas y ejecución
- Claude Haiku 4.5 para archivado (tareas simples de cierre)

Commands

Makefile

bash

```bash
make build              # Rust + Go
make build-rust         # Solo Rust release → dpf/target/release/dpf
make build-rust-static  # Binario estático musl
make build-go           # Solo Go → dpf-example
make test               # Verifica caps
make clean
```

Rust (cd dpf)

bash

```bash
cargo build --release
cargo check
cargo clippy -- -D warnings
cargo fmt
cargo run --release -- caps
```

Go (cd go-bridge)

bash

```bash
go build ./...
go mod tidy
cd example && go run .
```

Protocolo JSON

Comunicación Go ↔ Rust vía stdin/stdout:

bash

```bash
# One-shot
dpf process --job '{"operation":"resize",...}'

# Batch
dpf batch --file jobs.json
```

```
# Streaming
dpf --stream  # Lee JSON línea por línea
```

📛 Naming Conventions

REGLA DE ORO: Todo código, comentarios, nombres de archivos y documentación técnica en **INGLÉS**.

🌍 Language Policy

Contexto	Idioma	Ejemplo
Código fuente	Inglés	`fn resize_image()`
Comentarios	Inglés	`// Resizes the input image`
Nombres de archivo	Inglés	`image_processor.rs`
README.md	**Inglés**	`README.md` en raíz y subproyectos
Documentación técnica	Inglés	`docs/` , comentarios, explicaciones
PRDs/Especificaciones	Inglés	`PRD/001-multimedia-suite.md`
Commits Git	Inglés	`feat: add image resize operation`
Comunicación con usuario	Español	Issues, PRs, discusiones

📁 File & Directory Naming

General Rules

- **kebab-case** para archivos y carpetas: `image-processor.rs` , `cli-commands/`
- **SIN mayúsculas** en nombres de archivo (excepto README, LICENSE)
- **Extensiones explícitas**: `.rs` , `.go` , `.md` , `.json`
- **NO usar**: espacios, acentos, caracteres especiales, `camelCase` o `snake_case` en paths

Directory Structure

text

```
devpixelforge/
├── dpf/                        # Rust crate (abreviatura de devpixelforge)
│   ├── src/
│   │   ├── image/              # Módulo imágenes
│   │   │   ├── resize.rs
│   │   │   ├── crop.rs
│   │   │   └── mod.rs
│   │   ├── video/              # Módulo video
│   │   ├── audio/              # Módulo audio
│   │   ├── cli/                # Command line interface
│   │   └── lib.rs
│   ├── tests/                  # Tests de integración
│   └── Cargo.toml
```

```
├── go-bridge/                    # Go FFI bindings
│   ├── pkg/
│   │   ├── image/                # Package image
│   │   └── dpf/                  # Package principal
│   └── cmd/
│       └── example/              # Ejemplo CLI
├── PRD/                          # Product Requirements Docs
│   ├── 001-multimedia-suite.md
│   └── 002-image-optimizations.md
├── docs/                         # Documentación adicional
└── scripts/                      # Scripts de utilidad
```

🦀 Rust Naming (src/)

Elemento	Convención	Ejemplo	Incorrecto
Functions	`snake_case`	`fn resize_image()`	`fn resizeImage()`
Variables	`snake_case`	`let output_path`	`let outputPath`
Constants	`SCREAMING_SNAKE_CASE`	`const MAX_WIDTH: u32`	`const maxWidth`
Structs	`PascalCase`	`struct ImageProcessor`	`struct image_processor`
Enums	`PascalCase`	`enum ResizeMode`	`enum resize_mode`
Traits	`PascalCase`	`trait Processor`	`trait processor`
Modules	`snake_case`	`mod image_utils`	`mod ImageUtils`
Generic params	`PascalCase` (single letter)	`<T, U>`	`<t, u>`
Lifetimes	`'snake_case`	`'a, 'input`	`'A, 'Input`
Type aliases	`PascalCase`	`type PixelBuffer`	`type pixel_buffer`

Rust Examples

rust

```rust
// Constants
const DEFAULT_QUALITY: u8 = 85;
const SUPPORTED_FORMATS: &[&str] = &["jpg", "png", "webp"];

// Structs
pub struct ImageProcessor {
    input_path: PathBuf,
    output_dir: PathBuf,
}

// Enums
pub enum ResizeMode {
    Fit,
    Fill,
    Limit,
```

```
    Scale,
}

// Functions
fn resize_image(input: &Path, width: u32, mode: ResizeMode) -> Result<Image, Error> {
    let mut processor = ImageProcessor::new(input);
    processor.set_width(width);
    processor.process()
}

// Traits
pub trait Processor {
    fn process(&self) -> Result<Output, Error>;
    fn validate(&self) -> bool;
}
```

🐹 Go Naming (go-bridge/)

Elemento	Convención	Ejemplo	Incorrecto
Functions	`PascalCase` (exported) / `camelCase` (private)	`func ProcessImage()` / `func validateInput()`	`func process_image()`
Variables	`camelCase`	`outputPath`	`output_path`
Constants	`PascalCase` (exported) / `camelCase` (private)	`const MaxFileSize`	`const MAX_FILE_SIZE`
Structs	`PascalCase`	`type ImageProcessor struct`	`type imageProcessor struct`
Interfaces	`PascalCase` (-er suffix)	`type Processor interface`	`type IProcessor`
Packages	`lowercase` (single word)	`package image`	`package imageUtils`
Files	`snake_case`	`image_processor.go`	`imageProcessor.go`
Acronyms	All caps (URL, HTTP, ID)	`URLParser`, `HTTPClient`	`UrlParser`, `HttpClient`

Go Examples

go

```go
// Package
package image

// Constants
const MaxFileSize = 10 * 1024 * 1024 // 10MB
const defaultQuality = 85

// Structs
type ImageProcessor struct {
    InputPath  string
    OutputDir  string
    Width      uint32
    Quality    uint8
```

```go
}

// Interfaces
type Processor interface {
    Process() error
    Validate() bool
}

// Functions (exported)
func NewImageProcessor(input string) *ImageProcessor {
    return &ImageProcessor{
        InputPath: input,
        Quality:   defaultQuality,
    }
}

func (p *ImageProcessor) Process() error {
    if !p.validate() {
        return errors.New("invalid input")
    }
    // processing logic
    return nil
}

// Functions (private)
func (p *ImageProcessor) validate() bool {
    return p.InputPath != "" && p.Width > 0
}
```

📝 JSON / Configuration

json

```json
{
  "operation": "resize",
  "input_path": "/tmp/image.png",
  "output_dir": "/tmp/output",
  "widths": [320, 640, 1024],
  "quality": 85,
  "maintain_aspect_ratio": true,
  "output_format": "webp"
}
```

- **Keys**: `snake_case` en JSON
- **Values**: strings en kebab-case para enums (`"webp"` , `"resize-mode-fit"`)

🏷️ CLI Commands & Flags

bash

```bash
# Commands: kebab-case
dpf image resize --input file.png
dpf video transcode --profile web-mid
dpf audio normalize --input podcast.mp3

# Flags: kebab-case, descriptive
--input-path, -i
```

```
--output-dir, -o
--quality, -q
--maintain-aspect-ratio
--skip-validation
```

🔄 Git Commits

Formato: `<type>: <description>`

Type	Uso	Ejemplo
`feat`	Nueva funcionalidad	`feat: add AVIF format support`
`fix`	Corrección de bug	`fix: handle empty EXIF data`
`refactor`	Reestructuración	`refactor: split processor into modules`
`docs`	Documentación	`docs: update CLI usage examples`
`test`	Tests	`test: add resize operation tests`
`chore`	Tareas de mantenimiento	`chore: update dependencies`
`perf`	Mejoras de performance	`perf: optimize image decoding`

❌ Avoid These Patterns

rust

```rust
// BAD - Spanish in code
fn redimensionar_imagen() {}

// BAD - Mixed case in files
ImageProcessor.rs
imageUtils.go

// BAD - Unclear abbreviations
fn proc_img() {}  // ¿process? ¿processor? ¿procedural?
fn h() {}         // ¿height? ¿handler? ¿helper?

// BAD - Type in name
struct ImageProcessorStruct {}
fn process_image_fn() {}

// BAD - Context redundancy
image.image_width  // ya está en contexto "image"
// GOOD: image.width
```

✅ Checklist de Naming

Antes de commitear, verificar:

- [ ] Todo código en inglés
- [ ] Archivos/carpetas en `kebab-case`
- [ ] Rust: funciones/variables `snake_case`, types `PascalCase`
- [ ] Go: exported `PascalCase`, private `camelCase`
- [ ] JSON keys en `snake_case`

- [ ] CLI flags en `kebab-case`
- [ ] Commits en inglés con prefijo correcto
- [ ] Sin abreviaciones crípticas
- [ ] Nombres descriptivos (self-documenting)